

Node.js / Express — Sintesi della lezione

1. Web API, CRUD ed entità

Una Web API permette di esporre attraverso HTTP delle funzioni che prima potevano esistere soltanto all'interno del nostro codice.

Nel frontend avevamo già utilizzato funzioni che restituivano dati, ad esempio numeri o stringhe casuali. Con una Web API queste funzioni possono interagire con l'esterno:

```
Client
  ↓ HTTP Request
Server
  ↓
Logica
  ↓
HTTP Response
  ↓
Client
```

Il contesto generale è quindi quello delle operazioni **CRUD**:

- Create
- Read
- Update
- Delete

2. Entità e relazioni

Quando lavoriamo con dei dati, spesso non abbiamo una sola risorsa isolata.

Esempio della pizzeria:

```
Pizza
Ingredienti
```

Pizza e ingredienti possono essere considerate due entità distinte, tra le quali esiste una relazione.

Una pizza può avere molti ingredienti e un ingrediente può essere presente in molte pizze:

Pizza ↔ Ingredienti

Questa è una relazione **molti-a-molti**.

Lo stesso concetto si può applicare, per esempio, a una biblioteca:

Libro
Autore
Genere
Capitolo

Un libro può avere più autori e un autore può avere più libri.

Un libro può appartenere a più generi.

Un libro può invece avere molti capitoli, mentre ogni capitolo appartiene a un solo libro.

Il concetto di **entità e relazioni** è quindi un modello mentale per rappresentare dati e rapporti tra dati. I database relazionali sono una possibile implementazione di questo modello, ma il modello concettuale non coincide necessariamente con un database relazionale.

Anche una struttura JSON può rappresentare entità e relazioni, pur usando una struttura gerarchica anziché quella tipica delle tabelle relazionali.

3. La struttura di una Web API Express

Una semplice applicazione Express parte dall'inizializzazione di Express.

```
const express = require("express");  
  
const app = express();
```

`express()` è una **factory function**: crea e restituisce un oggetto che rappresenta l'applicazione Express.

Il nome `app` è soltanto una convenzione. Potremmo chiamarlo anche:

```
const server = express();
```

Il concetto importante non è il nome della variabile, ma l'oggetto Express che viene creato.

4. `server.listen()` e la porta

L'applicazione prende effettivamente vita quando viene messa in ascolto:

```
app.listen(3000);
```

`listen()` mantiene il processo attivo e in ascolto su una determinata porta.

Senza `listen()`, Node esegue semplicemente lo script:

```
import
↓
esecuzione
↓
fine dello script
↓
processo terminato
```

Con `listen()`:

```
import
↓
esecuzione
↓
server in ascolto
↓
processo attivo
```

Questa è la differenza fondamentale tra uno script che viene eseguito e un server che rimane attivo.

5. Le porte

Una porta identifica un punto attraverso il quale il sistema può ricevere comunicazioni di rete.

Esistono 65.536 porte, numerate da:

```
0 → 65535
```

Alcune sono tradizionalmente associate a determinati servizi.

Esempi citati nella lezione:

20/21	FTP
22	SSH
25	SMTP
80	HTTP
443	HTTPS

Durante lo sviluppo è comune utilizzare porte come:

3000
4000
5000

Purché non siano già occupate da un altro servizio.

6. Middleware ed endpoint

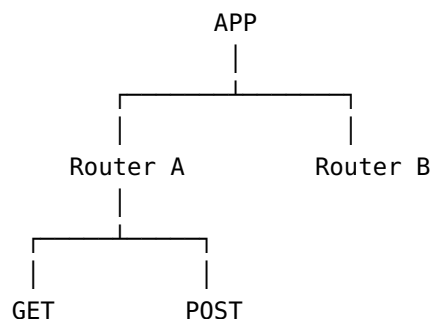
Una Web API Express viene costruita principalmente aggiungendo:

- middleware;
- endpoint.

I middleware vengono attraversati dalle richieste prima di arrivare eventualmente all'endpoint finale.

Un router è anch'esso un **middleware** dal punto di vista di Express.

Possiamo immaginare l'applicazione come una struttura ad albero:



La richiesta entra dall'applicazione e viene progressivamente filtrata fino ad arrivare all'endpoint corretto.

7. Router

Il router serve ad aggregare una parte delle rotte dell'applicazione.

Per esempio:

```
const router = express.Router();
```

e poi:

```
router.get("/", ...);  
router.post("/", ...);  
router.get("/:id", ...);
```

Il router può quindi rappresentare una determinata risorsa, per esempio:

```
/pizzas
```

L'app principale può poi montarlo:

```
app.use("/pizzas", router);
```

Il router è quindi un modo per organizzare e separare le rotte.

8. Endpoint e Controller

Un endpoint riceve principalmente:

```
request  
response
```

e gestisce la richiesta.

Per esempio:

```
router.get("/", (req, res) => {  
  // ...  
});
```

Questa funzione è un **request handler**, cioè un gestore della richiesta.

Quando l'applicazione cresce, possiamo spostare questa logica in un controller.

Struttura:

```
Router  
↓  
Controller
```

Il router si occupa della rotta.

Il controller si occupa della logica relativa alla richiesta e alla risposta.

Per esempio:

```
GET /pizzas/5  
↓  
Pizza Router  
↓  
Pizza Controller  
↓  
Pizza Repository
```

9. Repository Pattern

Il repository serve a separare la logica che lavora sui dati dalla logica del controller.

Il controller non dovrebbe preoccuparsi di sapere **come** vengono recuperati i dati.

Per esempio:

```
const pizza = repository.getById(id);
```

Al controller interessa ottenere la pizza.

Non gli interessa se il repository:

- legge un file;
- usa un database;
- usa un array in memoria;
- utilizza un'altra tecnologia.

La responsabilità dell'accesso ai dati viene quindi isolata nel repository.

Struttura:

```
Router
  ↓
Controller
  ↓
Repository
  ↓
Dati
```

Questa separazione permette di dividere le competenze.

10. CRUD nei diversi livelli

Le operazioni CRUD ricompaiono nei diversi livelli dell'applicazione.

A livello HTTP possiamo avere:

```
GET
POST
PUT
PATCH
DELETE
```

Esempio:

```
GET    /pizzas
GET    /pizzas/:id
POST   /pizzas
PUT    /pizzas/:id
PATCH /pizzas/:id
DELETE /pizzas/:id
```

Il router identifica la richiesta.

Il controller la gestisce.

Il repository esegue l'operazione sui dati.

11. I dati della richiesta

Una richiesta HTTP può contenere informazioni in punti diversi.

Tra quelli utilizzati nella lezione:

```
req.params  
req.query  
req.body
```

params

Servono principalmente per identificare una risorsa nella rotta.

```
GET /pizzas/5
```

Qui:

```
req.params.id
```

vale:

```
5
```

query

Sono parametri presenti nella query string.

Esempio:

```
GET /pizzas?name=margherita
```

body

Permette di inviare dati strutturati alla richiesta.

È particolarmente importante per:

```
POST
PUT
PATCH
```

12. Perché utilizzare JSON

Un modo possibile per inviare dati è utilizzare coppie chiave-valore come quelle dei form.

Il problema è che una semplice rappresentazione testuale non permette di distinguere bene i tipi.

Per esempio:

```
"154"
```

potrebbe rappresentare una stringa contenente cifre oppure un numero.

JSON permette invece di rappresentare anche il tipo del valore:

```
{
  "name": "Margherita",
  "price": 8,
  "available": true
}
```

Possiamo quindi distinguere:

```
string
number
boolean
null
array
object
```

JSON permette quindi di inviare un payload strutturato e complesso.

13. Body Parsing

C'è però un passaggio fondamentale.

Express, di default, non mette automaticamente il body della richiesta dentro:

```
req.body
```

È necessario effettuare il **body parsing**.

Per JSON possiamo utilizzare:

```
app.use(express.json());
```

Questo middleware permette a Express di leggere il body JSON e trasformarlo in un oggetto JavaScript.

Il processo concettuale è:

```
HTTP Request
  ↓
Body
  ↓
JSON
  ↓
JavaScript Object
  ↓
req.body
```

14. Perché Express non lo fa automaticamente?

Il body può essere molto grande.

Può contenere, per esempio:

```
un array di oggetti
un payload molto grande
molti dati
```

La lettura del body avviene attraverso uno **stream di byte**.

Il server deve:

1. ricevere i dati;
2. accumularli;
3. completare la lettura;
4. interpretarli;
5. trasformare il JSON in un oggetto JavaScript.

Per questo Express lascia all'applicazione la scelta di utilizzare o meno il body parser.

Non tutte le richieste hanno necessariamente bisogno di un body JSON.

15. POST, PUT e PATCH

La lezione distingue chiaramente i diversi tipi di operazione.

POST

Serve per creare una nuova risorsa.

```
POST /pizzas
```

Il client invia il payload della nuova pizza.

PUT

Serve per un aggiornamento completo.

```
PUT /pizzas/5
```

Ci aspettiamo l'intera pizza.

PATCH

Serve per un aggiornamento parziale.

```
PATCH /pizzas/5
```

Possiamo inviare soltanto le proprietà da modificare.

DELETE

Serve per eliminare una risorsa.

```
DELETE /pizzas/5
```

Non necessita normalmente di un body.

16. Validazione del body

Una volta ottenuto:

```
req.body
```

non possiamo semplicemente fidarci dei dati ricevuti.

Dobbiamo verificare che il body abbia la struttura prevista.

Per esempio, una pizza potrebbe richiedere:

```
{
  name: "4 formaggi",
  image: "pizza.jpg",
  ingredients: [
    "mozzarella",
    "gorgonzola"
  ]
}
```

Dobbiamo controllare:

- quali proprietà sono presenti;
 - il tipo di ogni proprietà;
 - la struttura degli array;
 - eventuali condizioni specifiche.
-

17. Modulo dedicato alla validazione

La validazione può essere separata dal controller in un modulo specifico.

Per esempio:

```
pizzas.validation.js
```

Possiamo avere tre validatori:

```
validateCreateBody  
validateUpdateBody  
validatePatchBody
```

Perché tre?

Perché le regole non sono uguali.

Create

Richiede:

```
name  
image  
ingredients
```

ma non `id`.

Update

Richiede tutte le proprietà previste.

Patch

Permette un sottoinsieme delle proprietà.

18. Validazione manuale

Una validazione può essere costruita inizialmente con normali condizioni JavaScript.

Per esempio:

```
if (typeof name !== "string") {  
    return null;  
}
```

Per l'ID:

```
if (typeof id !== "number") {  
    return null;  
}
```

Per l'immagine:

```
if (typeof image !== "string") {  
    return null;  
}
```

Se una condizione non viene rispettata:

```
return null
```

indica che la pizza non è valida.

19. Validazione degli array

Per `ingredients` non basta verificare che sia un array.

Bisogna anche controllare che gli elementi siano stringhe.

Qui entrano in gioco:

```
some()  
every()
```

`every()`

Restituisce `true` se tutti gli elementi rispettano il predicato.

```
ingredients.every(  
  ingredient => typeof ingredient === "string"  
);
```

`some()`

Restituisce `true` se almeno un elemento soddisfa il predicato.

Possiamo quindi scrivere:

```
ingredients.some(  
  ingredient => typeof ingredient !== "string"  
);
```

e se il risultato è `true`, abbiamo trovato almeno un ingrediente non valido.

20. Short-circuit

`some()` può essere più efficiente in questo caso perché si ferma quando trova il primo elemento che non rispetta la condizione.

Esempio:

```
["mozzarella", "pomodoro", 10, "basilico"]  
                        ↑  
                    primo errore
```

`some()` può fermarsi immediatamente.

Con `every()`, invece, stiamo chiedendo se **tutti** gli elementi sono validi.

Il concetto importante è quindi:

```
some()  
→ almeno uno  
  
every()  
→ tutti
```

21. Costruire l'oggetto validato

Un approccio consiste nel prendere dal body solamente le proprietà che ci interessano:

```
const { name, image, ingredients } = body;
```

Dopo averle validate, possiamo costruire un nuovo oggetto:

```
return {  
  name,  
  image,  
  ingredients  
};
```

Questo ha un vantaggio importante.

Se il client invia:

```
{  
  name: "Margherita",  
  image: "pizza.jpg",  
  ingredients: [],  
  pippo: "ciao"  
}
```

la proprietà:

```
pippo
```

non viene riportata nell'oggetto finale.

Le proprietà non previste vengono quindi scartate.

22. Due approcci al validatore

Il docente presenta due possibilità.

Approccio 1

Il validatore risponde solamente alla domanda:

```
È valido?
```

quindi:

```
true / false
```

Il controller mantiene il body originale.

Approccio 2

Il validatore utilizza il body per costruire l'oggetto validato:

```
body
↓
validator
↓
oggetto valido
```

oppure:

```
body
↓
validator
↓
null
```

Il secondo approccio viene preferito perché è più flessibile.

Non si limita a dire "sì/no", ma può anche:

- selezionare le proprietà;
- trasformare i dati;
- costruire un nuovo oggetto.

23. Validazione del PATCH

Il `PATCH` è particolare perché le proprietà sono facoltative.

Se arriva:

```
{  
  name: "4 formaggi"  
}
```

è corretto.

Non possiamo quindi applicare la stessa regola del `PUT`, dove tutte le proprietà devono essere presenti.

La logica diventa:

```
proprietà assente  
→ OK  
  
proprietà presente  
→ deve avere il tipo corretto
```

Per esempio:

```
name assente → OK  
name presente ma number → ERRORE  
name presente e string → OK
```

Lo stesso vale per `image` e `ingredients`.

24. L'ID

Nel `POST` l'ID non viene normalmente fornito dal client.

Il client invia:

```
name
image
ingredients
```

Il repository genera l'ID.

Concettualmente:

```
Client
  ↓
Pizza senza id
  ↓
Repository
  ↓
Generazione id
  ↓
Pizza con id
```

Durante un `PUT` o `PATCH`, invece, l'ID serve per individuare la risorsa da modificare.

25. Errori di validazione

Se il body non è valido, il controller può rispondere con:

```
400 Bad Request
```

Per esempio:

```
if (!pizza) {
  return res.status(400).json({
    error: "Pizza non valida"
  });
}
```

È importante distinguere questo caso da un errore come:

```
404 Not Found
```

Il `400` riguarda una richiesta non valida.

Il `404` può invece indicare che la risorsa richiesta non è stata trovata.

26. Sintassi JSON e validazione

Non tutti gli errori sono errori di validazione dell'applicazione.

Può esistere anche un errore **sintattico** del JSON.

Per esempio, se il JSON inviato non è formalmente valido, il parsing può fallire prima ancora che la nostra validazione della pizza venga eseguita.

Quindi possiamo distinguere:

```
JSON non valido
    ↓
errore di parsing
```

da:

```
JSON valido
    ↓
pizza con struttura/tipi errati
    ↓
errore di validazione
```

27. Librerie di validazione

La validazione manuale con tanti `if` funziona, ma può diventare molto lunga.

Le librerie di validazione permettono di descrivere uno **schema**.

Per esempio, concettualmente:

```
{
  name: string,
  image: string,
  ingredients: ...
}
```

Lo schema descrive la struttura che ci aspettiamo.

La libreria esegue internamente i controlli che noi prima scrivevamo manualmente.

Quindi:

PRIMA

```
if (...)  
if (...)  
if (...)  
if (...)  
if (...)
```

DOPO

```
schema  
  ↓  
validator  
  ↓  
risultato
```

Il vantaggio è avere una descrizione più dichiarativa della struttura prevista.

28. Repository su file e repository in memoria

Il repository può essere implementato in modi diversi.

Una prima versione può lavorare con un file:

```
PizzaFileRepository  
  ↓  
  JSON file
```

Una seconda versione può lavorare direttamente in memoria:

```
PizzaMemoryRepository  
  ↓  
  array
```

Il repository in memoria non ha bisogno di:

```
fs.readFile()  
fs.writeFile()
```

perché i dati vengono mantenuti direttamente in un array.

29. Perché rendere intercambiabili i repository?

L'idea è che il controller non debba sapere dove sono memorizzati i dati.

Possiamo avere:

```
Controller  
  ↓  
Repository  
  ↓  
File
```

oppure:

```
Controller  
  ↓  
Repository  
  ↓  
Memory
```

Il resto dell'applicazione può continuare a funzionare allo stesso modo.

Questo è possibile proprio grazie alla separazione delle responsabilità.

30. Configurazione dall'esterno

Un altro problema nasce quando una configurazione è scritta direttamente dentro un modulo.

Per esempio:

```
const path = "../data/pizzas.json";
```

Il percorso è **cablato** nel modulo.

Non possiamo controllarlo dall'esterno.

Possiamo invece trasformarlo in un parametro:

```
createPizzaRepository(path);
```

In questo modo il percorso diventa una configurazione esterna.

Concettualmente:

```
Configurazione
  ↓
Repository
  ↓
Dati
```

La configurazione può quindi essere centralizzata nella "spina dorsale" del server.

31. Introduzione alla Dependency Injection

Da questo concetto si arriva alla **Dependency Injection**.

L'idea iniziale è semplice:

invece di far decidere direttamente a un modulo quale implementazione utilizzare, gli forniamo dall'esterno ciò di cui ha bisogno.

Per esempio:

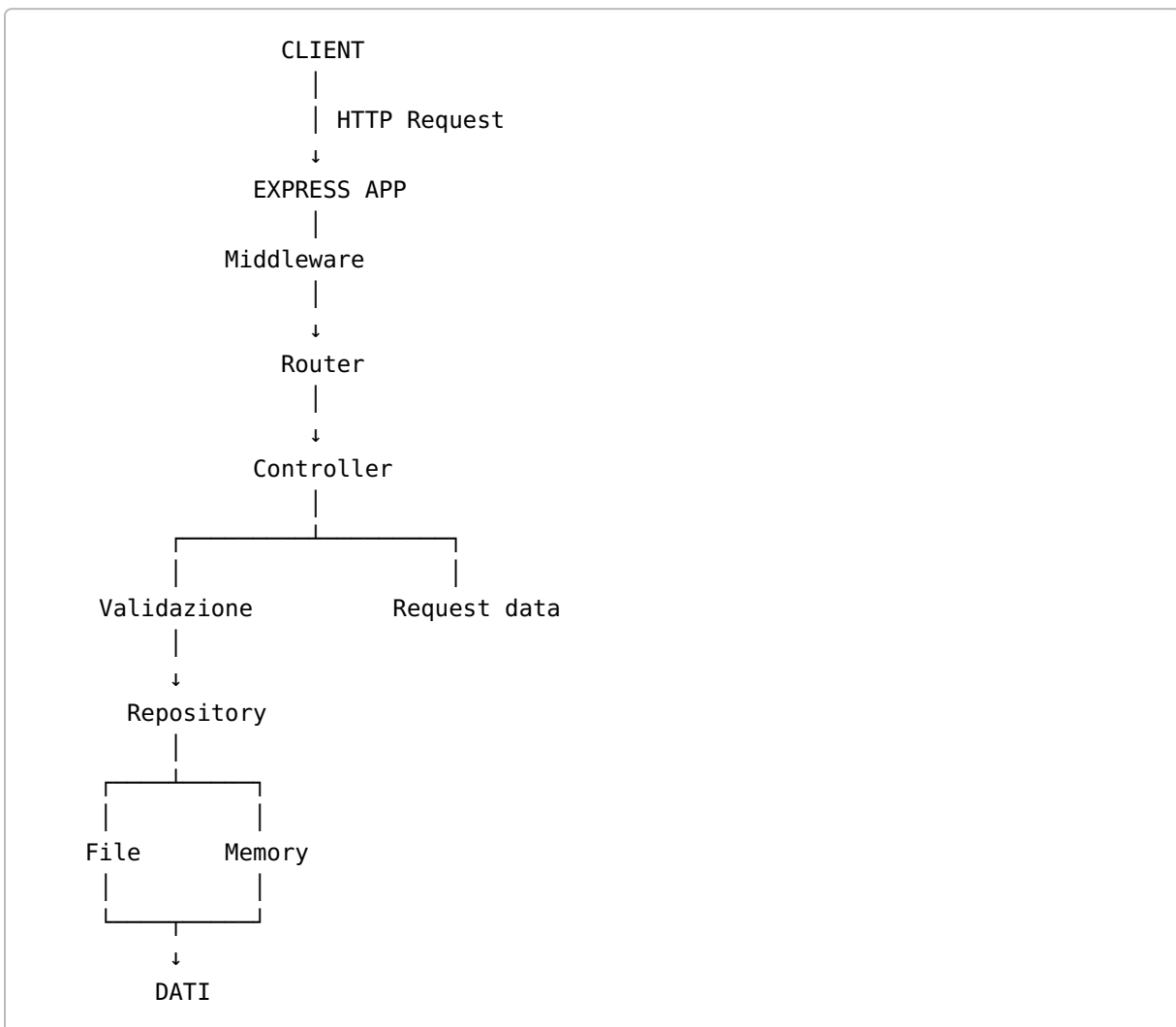
```
Server
  ↓
sceglie il repository
  ↓
lo passa al controller/servizio
```

Questo permette di rendere le implementazioni più intercambiabili.

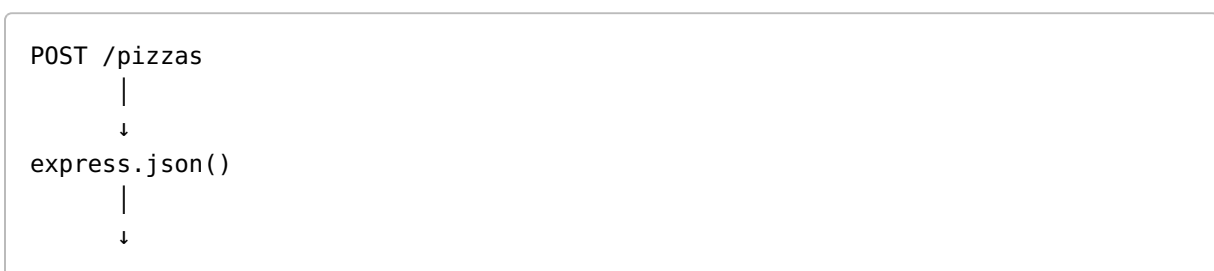
Nella lezione questo concetto viene introdotto, ma non viene ancora sviluppato completamente.

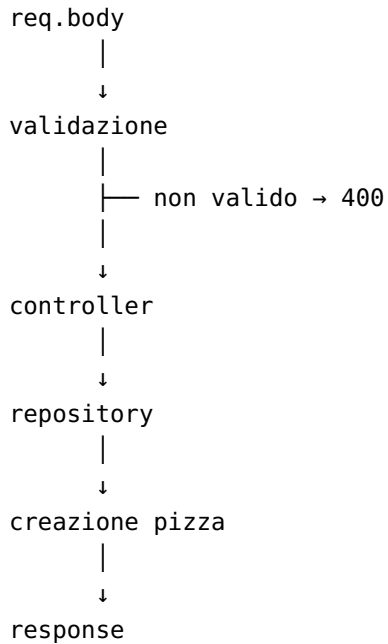
Schema mentale dell'intera lezione

Alla fine possiamo vedere l'intera Web API così:



E per una richiesta con body:





★ Cosa ricordare della lezione

Se dobbiamo ridurre tutto a pochi concetti fondamentali:

1. Express

```
const app = express();  
app.listen(3000);
```

`express()` crea l'applicazione, `listen()` mantiene il server in ascolto.

2. Router

Organizza le rotte e può essere montato come middleware.

3. Controller

Gestisce la richiesta e la risposta.

4. Repository

Gestisce l'accesso ai dati e nasconde al controller i dettagli della persistenza.

5. Body

Contiene il payload della richiesta.

6. `express.json()`

Permette di trasformare il JSON ricevuto nel `req.body`.

7. Validazione

Controlla che il body rispetti la struttura prevista.

8. CRUD

```
POST   → Create
GET    → Read
PUT    → Update completo
PATCH → Update parziale
DELETE → Delete
```

9. `some()` / `every()`

```
some() → almeno uno
every() → tutti
```

10. Separazione delle responsabilità

```
Router
  ↓
Controller
  ↓
Validation / Repository
  ↓
Dati
```

È questo il filo conduttore di tutta la lezione: **non mettere tutta la logica nello stesso posto, ma dividere il lavoro tra componenti con responsabilità precise.**